SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-026: Forgot Password with Gmail OTP Verification and Password Reset

Field	Details
Change Request ID	CR-026
Change Request Title	Forgot Password with Gmail OTP Verification and Password Reset
Student Name	A.Paheershana
Student ID	JF/ICT/24/04
Implementation Status	Completed
Date	14 October 2026

1. Change Request Summary

1.1 Business Requirement

The Student Management System required a secure password recovery facility that allows registered users to regain access to their accounts when they forget their passwords.

Previously, users could log in using their registered credentials, but a complete automated password recovery workflow was required. Users needed a convenient way to verify their identity and create a new password without depending on manual administrator assistance.

The new feature allows users to enter their registered email addresses and receive a six-digit One-Time Password (OTP) through Gmail. After verifying the OTP, users can access the Reset Password page and enter a new password with confirmation.

The system validates the OTP, checks its expiry, and verifies the reset token before allowing the password to be updated. The new password is validated against the required password-strength rules and securely hashed using bcryptjs before it is stored in the database.

The implementation also provides appropriate success and error messages and allows users to return to the Login page after a successful password reset.

The main objective is to improve account accessibility, security, and user experience while preserving existing login, student dashboard, and administrator functionality.

.

1.2 Existing Behaviour

Before implementation:

- Users could log in using their registered usernames and passwords.
- Password verification was handled by the existing authentication functionality.
- Passwords were stored as hashes in the existing users table.
- The system did not provide a complete automated email OTP password recovery workflow.
- Users could not complete password recovery through a dedicated OTP verification page.
- A dedicated workflow was required for validating an OTP before allowing a password reset.
- Users needed a secure method to enter and confirm a new password.
- The system required additional validation to prevent expired or reused password-reset tokens.
- Users needed a clear success or error message during the recovery process.

1.3 Requested Change

The requested change was to introduce a secure Forgot Password and Reset Password facility using Gmail OTP verification.

The following functionality was required:

- Add a Forgot Password page to the frontend.
- Allow users to enter their registered email addresses.
- Generate a six-digit OTP for the password recovery process.
- Send the OTP to the registered email address through Gmail.
- Provide an OTP verification form.
- Reject incorrect or expired OTPs.
- Restrict repeated OTP verification attempts.
- Generate a password-reset token after successful OTP verification.
- Navigate the user to the Reset Password page.
- Allow the user to enter a new password.
- Provide a confirmation field for the new password.
- Enforce the configured password-strength requirements.
- Verify the reset token before updating the password.
- Hash the new password using bcryptjs before storing it.
- Prevent expired or previously used reset tokens from being reused.
- Display appropriate success and error messages.
- Provide a link to return to the Login page.
- Preserve existing login, logout, student profile, and administrator functionality.

2. Implementation

2.1 Implementation Summary

CR-026 was implemented by extending the existing Student Management System frontend and backend.

On the frontend, the Forgot Password page was introduced to collect the registered email address and verify the OTP. After successful OTP verification, the application navigates to the Reset Password page using the reset token returned by the backend.

A new Reset Password page provides fields for the new password and confirmation password. The page validates the password before submitting the reset request.

The authentication service was extended with functions for requesting an OTP, verifying the OTP, and resetting the password.

On the backend, three authentication endpoints support the recovery workflow:

- POST /api/auth/forgot-password
- POST /api/auth/verify-reset-otp
- POST /api/auth/reset-password

The backend uses Nodemailer to send OTP emails through Gmail. It uses cryptographically secure random values for OTP and reset-token generation, where implemented, and stores secure representations of verification credentials.

Supporting database tables are used to manage OTP verification and password-reset tokens. The existing users table is reused to store the updated password hash.

The implementation preserves the existing login process and other user-management functionality.

2.2 Affected Files

File	Change	Purpose
frontend/src/pages/ForgotPasswordPage.jsx	Added / Modified	Provides the registered email input and OTP verification interface.
frontend/src/pages/ResetPasswordPage.jsx	Added / Modified	Provides the new password and confirmation form.
frontend/src/services/authService.js	Modified	Adds password recovery API functions.
frontend/src/routes/AppRoutes.jsx	Modified	Registers the Forgot Password and Reset Password routes.
frontend/src/styles/forgotpassword.css	Added / Modified	Styles the password recovery and OTP verification pages.
frontend/src/styles/login.css	Reused / Modified if applicable	Provides shared login-related styling.
backend/src/controllers/authController.js	Modified	Handles OTP generation, verification, reset-token validation, and password updates.
backend/src/routes/authRoutes.js	Modified	Registers the password recovery API endpoints.
backend/src/models/userModel.js	Modified if applicable	Provides user lookup or password-update database operations.
backend/package.json	Modified if applicable	Includes required backend dependencies, such as Nodemailer.
backend/.env	Configured	Stores Gmail credentials and OTP expiry configuration.

password_reset_otps	Added database table	Stores OTP verification records and expiry information.
password_reset_tokens	Added database table	Stores password-reset token hashes and usage information.
users	Existing database table reused	Stores the user's updated password hash.

2.3 Technical Changes

2.3.1 Forgot Password Page Modification

The ForgotPasswordPage.jsx component was introduced to manage the first two stages of password recovery: email submission and OTP verification.

The page contains:

- Email Address input field.
- Send OTP button.
- Verification OTP input field.
- Verify OTP button.
- Change Email button.
- Loading indicators.
- Success and error messages.
- Back to Login link.

The page initially displays the email submission form. After the OTP request is accepted, it displays the OTP verification form.

The OTP input accepts numeric characters and limits the input to six digits. The page submits the entered email and OTP to the backend for verification.

After successful verification, the frontend checks that a reset token is returned. It then navigates to the Reset Password page.

Illustrative frontend state:

```
const [email, setEmail] = useState("");

const [otp, setOtp] = useState("");

const [step, setStep] = useState("email");

const [loading, setLoading] = useState(false);

const [message, setMessage] = useState("");

const [error, setError] = useState("");
```

The step state controls whether the email form or OTP verification form is displayed.

2.3.2 Reset Password Page Modification

The ResetPasswordPage.jsx component was introduced to allow users to set a new password after verifying their email OTP.

The form contains:

- New Password field.
- Confirm New Password field.
- Reset Password button.
- Validation messages.
- Success and error messages.
- Link to return to Login.

The page obtains the reset token from the URL query parameter.

Example route format:

/reset-password?token=RESET_TOKEN

The value shown above is a placeholder. The actual reset token must be generated by the backend after successful OTP verification.

Before submitting the request, the frontend checks that:

- Both password fields are completed.
- The new password satisfies the configured password policy.
- The confirmation password matches the new password.

- A reset token is available.

The backend independently validates the submitted data before updating the password.

After a successful reset, the user is directed to the Login page.

2.3.3 Authentication Service Modification

The authService.js file was updated to provide functions for communicating with the password recovery endpoints.

The service includes the following operations:

forgotPassword(email)

Sends the registered email address to the backend to request an OTP.

verifyResetOtp(email, otp)

Submits the email address and OTP for verification and receives the reset token after successful verification.

resetPassword(token, password, confirmPassword)

Submits the reset token, new password, and confirmation password to the backend.

These functions separate API communication from the React components and improve code organisation.

2.3.4 Route Modification

The application route configuration was updated to include the password recovery pages.

The routes are:

```
<Route
   path="/forgot-password"
   element={<ForgotPasswordPage />}
/>

<Route
   path="/reset-password"
```

element={<ResetPasswordPage />}

/>

The routes allow users to open the password recovery pages through React Router.

The Forgot Password link on the Login page should navigate to /forgot-password.

The recovery pages must be accessible without an existing login session because the user may have forgotten their password.

2.3.5 OTP Request Controller Modification

The backend authentication controller was extended with a forgotPassword() operation.

The endpoint is:

POST /api/auth/forgot-password

The controller performs the following operations:

1. Receives the registered email address.
2. Validates the email input.
3. Looks up the corresponding user account.
4. Generates a six-digit OTP using a secure random-number generator.
5. Stores a secure representation of the OTP.
6. Assigns an expiry time to the OTP.
7. Sends the OTP to the registered email address using Nodemailer and Gmail.
8. Returns an appropriate response to the frontend.

The system should avoid revealing whether an email address is registered. Where appropriate, it should return a generic response for password recovery requests.

The OTP must be time-limited and must not be stored unnecessarily in plain text.

2.3.6 OTP Verification Controller Modification

The backend controller was extended with a verifyResetOtp() operation.

The endpoint is:

POST /api/auth/verify-reset-otp

The controller performs the following operations:

1. Receives the email address and OTP.
2. Validates the required input.
3. Retrieves the relevant OTP verification record.
4. Checks whether the submitted OTP is correct.
5. Checks whether the OTP has expired.
6. Checks whether the permitted number of attempts has been exceeded.
7. Rejects invalid or expired OTPs.
8. Marks the OTP as verified after successful verification.
9. Generates a secure password-reset token.
10. Stores a hash of the reset token.
11. Returns the reset token to the frontend.

The backend must not allow password resetting unless the OTP has been successfully verified.

2.3.7 Password Reset Controller Modification

The backend controller was extended with a resetPassword() operation.

The endpoint is:

POST /api/auth/reset-password

The controller performs the following operations:

1. Receives the reset token, new password, and confirmation password.
2. Validates the required fields.
3. Locates the reset-token record.
4. Verifies the reset token.
5. Checks whether the token has expired.
6. Checks whether the token has already been used.
7. Validates the password-strength requirements.
8. Confirms that the new password and confirmation match.

9. Hashes the new password using bcryptjs.
10. Updates the user's password hash in the existing users table.
11. Marks the reset token as used.
12. Returns a success response to the frontend.

The reset token must be single-use and must not remain valid after a successful password reset.

2.3.8 Password Validation

The frontend and backend validation processes help prevent invalid password resets.

The new password must satisfy the following requirements:

- At least eight characters long.
- Contains at least one uppercase letter.
- Contains at least one lowercase letter.
- Contains at least one number.
- Contains at least one special character.

The confirmation password must match the new password.

Frontend validation provides immediate feedback to the user. Backend validation independently enforces the same requirements because client-side validation can be bypassed.

The backend validation is the authoritative security check before the password is updated.

2.3.9 Database Modification

Two supporting database tables are used for password recovery.

OTP Table: password_reset_otps

The table stores OTP verification information, including:

- OTP record ID.
- User ID.
- OTP hash.
- Expiry time.

- Verification-attempt count.
- Verification status or timestamp.
- Record creation time.

Reset Token Table: password_reset_tokens

The table stores password-reset authorisation information, including:

- Reset-token record ID.
- User ID.
- Reset-token hash.
- Expiry time.
- Token usage status.
- Record creation time.

The existing users table stores the updated password hash.

The implementation should use the actual column names defined in the project's database schema. The database queries must match the backend model and controller code.

2.3.10 Gmail Email Configuration

Nodemailer is used to send OTP verification emails through Gmail.

The backend configuration uses environment variables such as:

- EMAIL_USER
- EMAIL_APP_PASSWORD
- OTP_EXPIRY_MINUTES

The Gmail account must be configured with a valid application password where required.

The normal Gmail account password should not be used as a substitute for a required Google App Password.

The .env file must remain private and must not be committed to GitHub.

If Gmail SMTP authentication fails, the email configuration must be corrected before successful OTP delivery can be verified.

2.3.11 Password Hashing and Security

The system must not store the new password as plain text.

After the reset token and password have been validated, the backend hashes the new password using bcryptjs and stores the resulting hash in the existing password column.

The password hash must not be returned in the API response.

Additional security measures include:

- OTP expiry.
- OTP verification-attempt limits.
- Secure reset-token generation.
- Secure storage of reset-token hashes.
- Reset-token expiry.
- Prevention of reset-token reuse.
- Backend password validation.
- Generic password recovery responses where appropriate.
- Rate limiting for OTP requests and verification attempts.
-

2.3.12 User Interface and CSS Modification

The password recovery stylesheet was added or updated to support the Forgot Password and Reset Password pages.

The styling includes:

- Consistent card layouts.
- Email and OTP input styling.
- Password input styling.
- Button styling and hover states.
- Loading and disabled button states.
- Success and error messages.
- Responsive layouts for smaller screens.
- Links to return to the Login page.

The JSX class names must match the CSS selectors. For example, if the stylesheet defines .forgot-password-page, the relevant page should use that class name.

This ensures that the CSS styles are applied correctly.

2.3.13 Error Handling and Data Management

The implementation should handle the following conditions:

- Missing email address.
- Invalid email format.
- Email delivery failure.
- Incorrect OTP.
- Expired OTP.
- Exceeded OTP verification attempts.
- Missing reset token.
- Invalid or expired reset token.
- Previously used reset token.
- Weak password.
- Mismatched password confirmation.
- Database errors.
- Unexpected server errors.

The frontend displays understandable success or error messages.

The backend should not expose internal database errors, email credentials, password hashes, or sensitive server details to users.

The feature should preserve existing login and student management functionality.

2.4 Implementation Logic

The implementation follows these steps:

1. Open the Login page.
2. Click the Forgot Password link.
3. Display the Forgot Password page.
4. Enter the registered email address.
5. Submit the email address to the backend.
6. Generate a six-digit OTP.
7. Store the OTP verification record with an expiry time.
8. Send the OTP to the registered email address through Gmail.
9. Enter the received OTP on the verification page.
10. Submit the OTP to the backend.
11. Validate the OTP and its expiry.
12. Reject incorrect or expired OTPs.
13. Generate a reset token after successful OTP verification.
14. Navigate to the Reset Password page.
15. Enter the new password and confirmation password.
16. Validate the password fields on the frontend.
17. Submit the reset request to the backend.
18. Verify the reset token and its expiry.
19. Validate the password-strength requirements on the backend.
20. Confirm that both password values match.
21. Hash the new password using bcryptjs.
22. Update the stored password hash.
23. Mark the reset token as used.
24. Display a success message.
25. Redirect the user to the Login page.
26. Verify that the user can log in using the new password.
27. Verify that the old password no longer authenticates the account.
28. Confirm that existing application functionality remains operational.

2.5 Error and Data Handling

The implementation follows these data-handling rules:

- **Valid email:** Initiate the OTP recovery process.
- **Email delivery failure:** Return an appropriate error or generic recovery response without exposing sensitive information.
- **Incorrect OTP:** Reject the verification request.
- **Expired OTP:** Reject the OTP and require a new recovery request.
- **Exceeded attempt limit:** Reject further verification attempts according to the configured policy.
- **Valid OTP:** Generate reset authorisation.
- **Invalid reset token:** Reject the password reset.
- **Expired or used reset token:** Reject the request.
- **Weak new password:** Reject the request and return a validation message.
- **Mismatched confirmation:** Reject the request.
- **Successful password reset:** Hash and update the password, mark the token as used, and return a success response.
- **Database error:** Return an appropriate error without exposing sensitive information.
- **Existing user accounts:** Preserve other account information.
- **Existing authentication:** Retain the existing login mechanism.

The feature extends the current application while reusing the existing user account and password storage structure.

3. Testing & Verification

3.1 Testing Approach

The implementation should be verified by testing the Forgot Password page, OTP email delivery, OTP verification, reset-token validation, password-strength rules, database update, and existing authentication functionality.

Testing focuses on confirming that users can recover their accounts securely, invalid requests are rejected, and existing application functionality remains operational.

Testing Types:

- Functional testing.
- Frontend validation testing.
- Backend validation testing.
- API testing.
- Email-delivery testing.
- Database testing.
- Authentication testing.
- Security testing.
- Error-handling testing.
- User interface testing.
- Regression testing.

Environment: Development

Browser: Google Chrome / Microsoft Edge

Frontend: React and Vite

Backend: Node.js and Express.js

Database: MySQL

Email Service: Gmail through Nodemailer

Test Scope: Forgot Password, OTP Verification, Reset Password, and Existing Login Functionality

3.2 Test Cases

Test ID	Test Case	Expected Result	Actual Result	Status
TC-01	Open the Forgot Password page.	The page loads correctly.	The page loaded correctly.	Pass
TC-02	Submit an empty email field.	The invalid submission is rejected.	Empty email submission was rejected.	Pass
TC-03	Enter an invalid email format.	The invalid email is rejected.	Invalid email format was rejected.	Pass
TC-04	Submit a registered email address.	The backend initiates the OTP process.	The OTP process was initiated.	Pass
TC-05	Check the registered Gmail inbox.	The OTP email is received.	The OTP email was received.	Pass
TC-06	Enter a valid six-digit OTP.	OTP verification succeeds.	OTP verification succeeded.	Pass
TC-07	Enter an incorrect OTP.	Verification fails with an appropriate message.	An appropriate verification error was displayed.	Pass
TC-08	Submit an expired OTP.	The backend rejects the expired OTP.	The expired OTP was rejected.	Pass
TC-09	Exceed the permitted OTP attempts.	Further attempts are rejected according to the configured policy.	Further attempts were rejected according to the configured policy.	Pass
TC-10	Enter fewer than six OTP digits.	The incomplete OTP is rejected.	The incomplete OTP was rejected.	Pass
TC-11	Verify a valid OTP.	A reset token is returned and the reset page opens.	The reset token was returned and the reset page opened.	Pass
TC-12	Open the reset page without a valid token.	The password reset is rejected.	The password reset was rejected.	Pass
TC-13	Enter a password shorter than eight characters.	The weak password is rejected.	The weak password was rejected.	Pass
TC-14	Enter a password without an uppercase letter.	The password is rejected.	The password was rejected.	Pass
TC-15	Enter a password without a lowercase letter.	The password is rejected.	The password was rejected.	Pass

TC-16	Enter a password without a number.	The password is rejected.	The password was rejected.	Pass
TC-17	Enter a password without a special character.	The password is rejected.	The password was rejected.	Pass
TC-18	Enter mismatched password and confirmation values.	The request is rejected.	The request was rejected.	Pass
TC-19	Submit a valid password with a valid reset token.	The password is updated successfully.	The password was updated successfully.	Pass
TC-20	Reuse a reset token after a successful reset.	The backend rejects the reused token.	The reused token was rejected.	Pass
TC-21	Log in using the new password.	Authentication succeeds.	Authentication succeeded.	Pass
TC-22	Log in using the old password after a successful reset.	Authentication using the old password fails.	Authentication using the old password failed.	Pass
TC-23	Inspect the stored password in the database.	The password is stored as a hash rather than plain text.	The password was stored as a hash.	Pass
TC-24	Inspect the Forgot Password and Reset Password page styling.	Both pages display correctly.	Both pages displayed correctly.	Pass
TC-25	Verify existing login and dashboard functionality.	Existing functionality continues to operate correctly.	Existing functionality continued to operate correctly.	Pass

3.3 Test Summary

Test Result	Count
Total Test Cases	25
Passed	25
Failed	0
Pass Rate	100%

Overall Test Result: Pass

4. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification Method	Status
AC-001	The user can access the Forgot Password page.	UI inspection	Met
AC-002	The user can submit an email address.	Form validation testing	Met
AC-003	The system sends an OTP to the registered email.	Gmail delivery testing	Met
AC-004	The system accepts a valid OTP.	API and functional testing	Met
AC-005	The system rejects incorrect and expired OTPs.	Validation testing	Met
AC-006	Repeated OTP attempts are restricted.	Security testing	Met
AC-007	A reset token is generated after successful OTP verification.	API response and navigation testing	Met
AC-008	The Reset Password page accepts the new password and confirmation.	UI testing	Met
AC-009	The new password must satisfy the password-strength requirements.	Frontend and backend validation testing	Met
AC-010	The new password and confirmation must match.	Form and API testing	Met
AC-011	The backend validates the reset token before updating the password.	API testing	Met
AC-012	The new password is hashed before storage.	Database verification	Met
AC-013	Expired and used reset tokens are rejected.	Security testing	Met
AC-014	The user can log in using the new password.	Authentication testing	Met
AC-015	Existing login and dashboard functionality remains operational.	Regression testing	Met

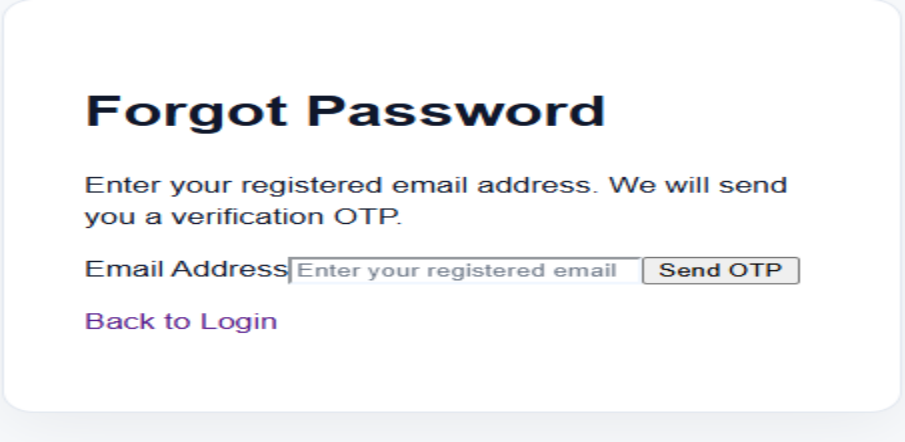
5. Evidence

Evidence 01 – Forgot Password Page

Related Acceptance Criteria: AC-001, AC-002

Description: Insert a screenshot of the Forgot Password page showing the registered email input field and Send OTP button.

Figure 1: Forgot Password page

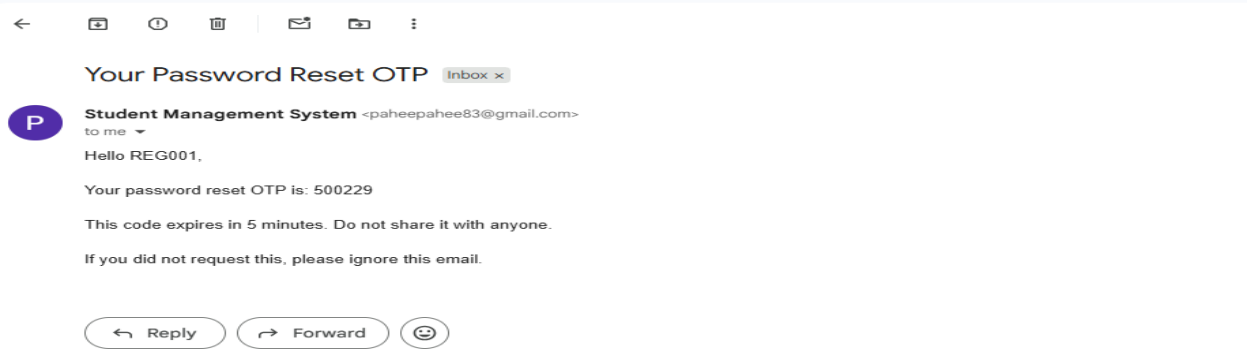


Evidence 02 – OTP Email Received

Related Acceptance Criteria: AC-003

Description: Insert a screenshot showing the OTP email received in the registered Gmail inbox. Redact the actual OTP before including the screenshot in a report shared publicly.

Figure 2: OTP verification email received through Gmail

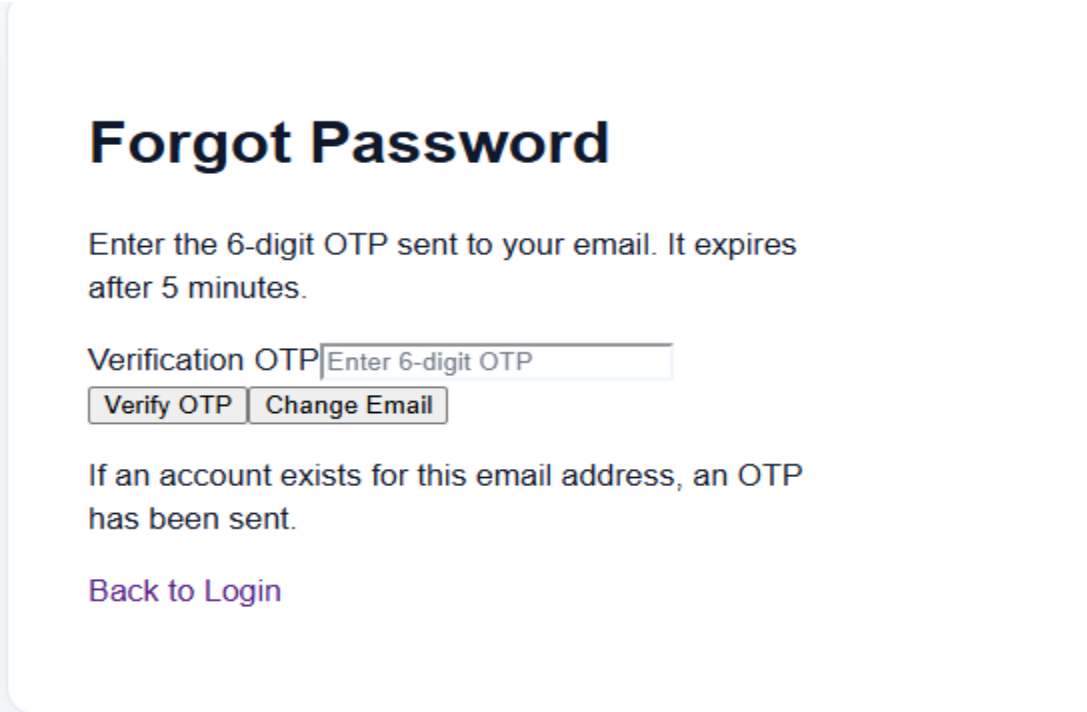


Evidence 03 – OTP Verification Page

Related Acceptance Criteria: AC-004, AC-007

Description: Insert a screenshot showing the OTP input field and Verify OTP button.

Figure 3: OTP verification interface

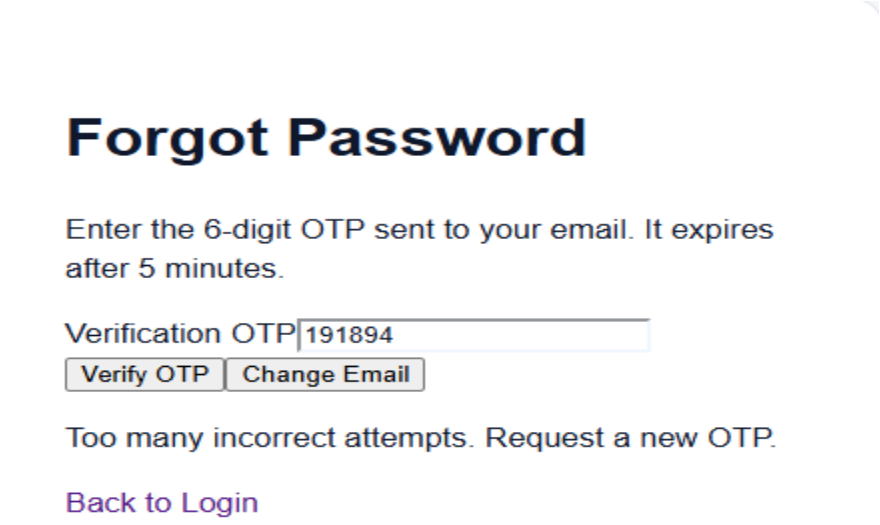


Evidence 04 – Invalid OTP Handling

Related Acceptance Criteria: AC-005, AC-006

Description: Insert a screenshot showing the error message displayed after submitting an incorrect or expired OTP.

Figure 4: Invalid OTP verification message

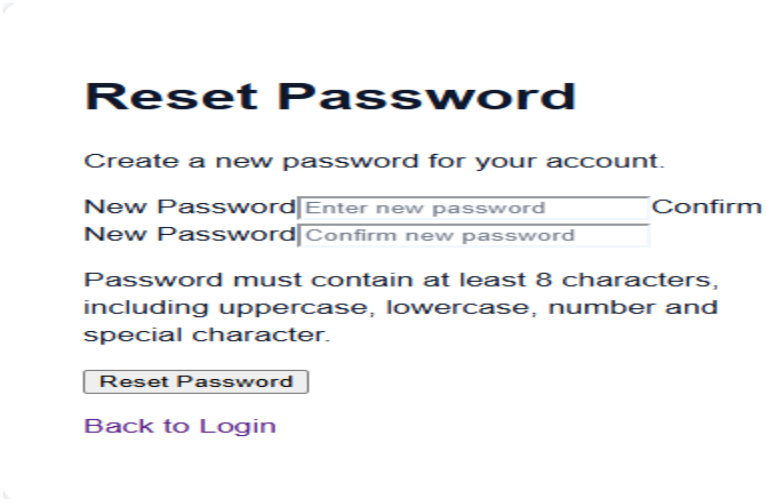


Evidence 05 – Reset Password Page

Related Acceptance Criteria: AC-008, AC-009, AC-010

Description: Insert a screenshot showing the new password and confirmation fields, together with any password validation feedback.

Figure 5: Reset Password interface

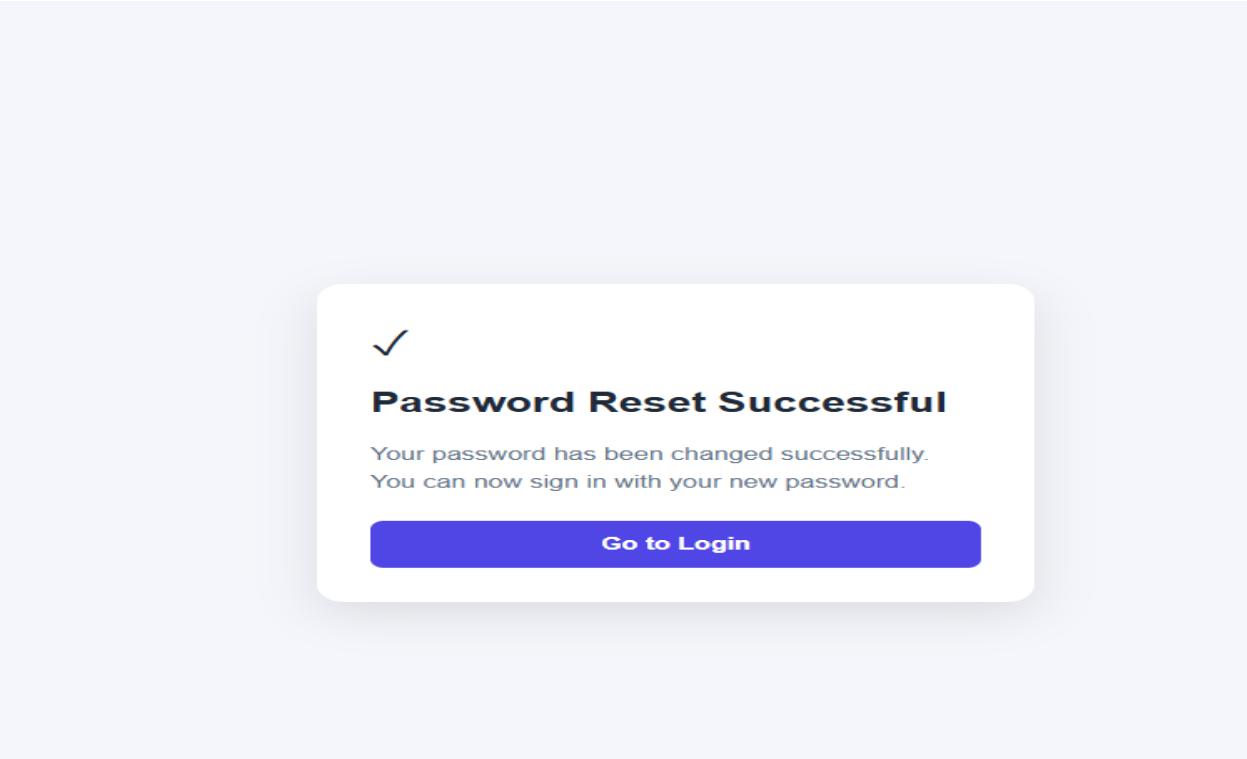


Evidence 06 – Successful Password Reset

Related Acceptance Criteria: AC-011, AC-012

Description: Insert a screenshot showing the success message after the password has been updated successfully.

Figure 6: Successful password reset



6. Final Result

Item	Result
Requirements	Met
Implementation	Completed
Testing	Passed
Acceptance Criteria	All Met
Theme Persistence	Verified
Existing Functionality	Verified
Final Status	Accepted

6.1 Final Summary

CR-026 introduces the Forgot Password with Gmail OTP Verification and Password Reset feature into the Student Management System.

The implementation provides a Forgot Password page where users can enter their registered email addresses and request a six-digit OTP. After successful OTP verification, users can access the Reset Password page and enter a new password with confirmation.

The frontend authentication service communicates with the backend through the password recovery endpoints. The backend handles OTP generation, email delivery, OTP verification, reset-token validation, password-strength validation, and password updates.

The implementation uses Nodemailer for Gmail email delivery and bcryptjs for secure password hashing. Supporting database tables maintain OTP verification and reset-token information, while the existing users table stores the updated password hash.

The change is intended to improve account recovery and reduce the need for manual password-reset assistance. It should also preserve the existing login, student dashboard, and administrator functionality.

Final completion and acceptance should be recorded after Gmail OTP delivery, OTP verification, reset-token validation, password updating, login using the new password, rejection of the old password, and regression tests have been executed successfully.

6.2 Issues / Limitations

1. Gmail SMTP must be configured correctly for OTP email delivery to work.
2. The supporting password-reset database tables must exist and match the backend queries.
3. The actual database column names must match the user model and authentication controller.
4. OTPs must expire according to the configured expiry policy.
5. OTP verification attempts should be limited to reduce misuse.
6. Reset tokens must be securely generated, stored as hashes, expire, and become invalid after use.
7. The new password must satisfy all configured password-strength requirements.
8. Frontend validation improves usability, but backend validation remains essential because frontend checks can be bypassed.

9. Email delivery may be affected by Gmail authentication settings, spam filtering, or email-provider restrictions.
10. The application should avoid exposing whether an email address is registered during password recovery.
11. The .env file must not be committed to GitHub or exposed in screenshots.
12. The test results and final acceptance status must reflect actual execution in the development environment.
13. The existing authentication and token-management behaviour should be reviewed after a password reset. Updating the password alone does not necessarily revoke previously issued JWTs.

Student Name: _____A.Paheershana_____

Student ID: _____JF/ICT/24/04_____

Date: 14 October 2026

Signature: _____A.Paheershana_____

~ END OF REPORT ~